KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Smart To-Do Task Manager

Submitted by

SHAMITHA J

Roll No.: 25BIT053

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: 2025 – 2026

APRIL 2026

Semester: II

DECLARATION

I hereby declare that the project entitled “Smart To-Do Task Manager” submitted in partial fulfillment of the requirements for the course Java Programming is a record of my original work carried out under the guidance of Dhileeban A, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

SHAMITHA J

CERTIFICATE

This is to certify that the project entitled “Smart To-Do Task Manager” is a bona fide work carried out by SHAMITHA J, B.Sc. Information Technology, KG College of Arts & Science, in partial fulfillment of the requirements for the course [Java Programming].

This project has been completed under my supervision and guidance during the academic period April 2026.

Place: Coimbatore

Date: 09.04.2026

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

_____

Head of the Department
(Technical Head, KGCAS)

_____

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Smart To-Do Task Manager is a desktop-based task management application developed using Java Swing. It provides a simple graphical interface for entering tasks, assigning priorities, and organizing daily activities. A user can add a task, choose High, Medium, or Low priority, mark a selected task as completed, or delete a task that is no longer needed. The application is intended to make basic task tracking convenient and accessible through a single window.

The application validates task entry and displays helpful dialog messages when a task is empty or when an action is attempted without selecting a task. Newly added tasks are sorted according to priority, with High priority tasks placed before Medium and Low priority tasks. The project uses Java Swing components such as JFrame, JLabel, JTextField, JComboBox, JButton, JList, DefaultListModel, JScrollPane, and JOptionPane.

The main objective is to provide a lightweight and easy-to-use task organizer that helps users record activities, recognize task importance, mark work as completed, and remove tasks when necessary. The interface keeps the main actions visible so that task management can be performed with minimal steps.

This project demonstrates event-driven programming, graphical user interface design, conditional logic, list-model operations, and basic sorting in Java. It can serve as a foundation for future enhancements such as saving tasks to a file or database, due dates, reminders, and search functionality.

1. INTRODUCTION

Managing daily activities becomes difficult when tasks are written in different places or remembered without a consistent system. A simple task manager provides a central place to record pending work and identify which activities should be handled first. A graphical interface can make this process easier by offering clear input fields and direct action buttons.

Java Swing provides GUI components that can be used to build desktop applications. Using these components, the Smart To-Do Task Manager presents a task-entry field, a priority selector, action buttons, and a scrollable task list. Event listeners respond to user actions and update the list based on the selected operation.

The Smart To-Do Task Manager allows a user to enter a task description and select its priority as High, Medium, or Low. The Add Task button validates the description, adds the task to the list, clears the input field, and sorts the list by priority. The Complete button marks a selected task as completed, while the Delete button removes a selected task from the list.

The application is implemented in Java using the Swing library and AWT event handling. It uses a JFrame for the main window, labels and input controls for task entry, a DefaultListModel to manage displayed items, and JOptionPane dialogs to communicate validation and action results.

The primary goal is to make everyday task organization straightforward while demonstrating how Java can be used to create a practical desktop GUI. The current application manages tasks in memory during execution; persistent storage is not implemented in the supplied source code.

1.1 Objective

The main objective of the Smart To-Do Task Manager is to develop a simple Java Swing application that helps users add, prioritize, complete, and delete tasks through a graphical interface.

The specific objectives of this project are:

- To create a desktop task manager using Java Swing components.
- To allow users to enter task descriptions and choose High, Medium, or Low priority.
- To validate task input and provide clear feedback through dialog messages.
- To sort tasks according to their selected priority.
- To support marking selected tasks as completed and deleting selected tasks.
- To demonstrate event handling, list models, conditional statements, and basic sorting in Java.

1.2 Overview

The Smart To-Do Task Manager is a Java Swing desktop application that displays a single window containing a task input field, a priority drop-down list, three action buttons, and a scrollable list of tasks.

When the user enters a non-empty task and selects Add Task, the task is added with its priority and the list is sorted. The Complete button prefixes a selected task with a completed label. The Delete button removes the selected item. If the input is empty or no task is selected, the application displays an appropriate message.

Tasks are stored in a DefaultListModel while the application is running. The current source code does not save task data permanently, so the list is not retained after the application is closed.

The project demonstrates how a small GUI application can support basic productivity and task organization using Java.

1.3 Features

The Smart To-Do Task Manager includes the following main features:

• Task Entry – Allows the user to type a task description in the input field.

• Priority Selection – Provides High, Medium, and Low priority options for each new task.

• Add Task – Validates the task field, adds the task to the list, clears the field, and displays a success message.

• Priority Sorting – Arranges tasks so that High priority appears before Medium and Low priority.

• Complete Task – Marks a selected task as completed and prevents it from being marked complete again.

• Delete Task – Removes a selected task from the list.

• Input Validation – Displays dialog messages when the task field is empty or no task is selected.

2. PROGRAM ANALYSIS

Program analysis identifies the purpose of the application, the user actions it must support, and the components needed to implement those actions. For this project, the analysis focuses on straightforward task entry, priority selection, task status updates, and deletion.

The application is designed to replace an informal, scattered approach to remembering tasks with a single graphical list. It keeps the controls visible and provides feedback for common user actions.

2.1 Problem Definition

Users may forget tasks, struggle to distinguish urgent work from less important activities, or find it inconvenient to update a handwritten list. A basic task organizer should provide a clear place to enter activities, assign priority, identify completed work, and remove items when they are no longer needed.

The main requirements identified are:

• Enter a task description.
• Choose a priority level.
• Add the task to a visible list.
• Sort tasks by priority.
• Mark a selected task as completed.
• Delete a selected task.
• Show feedback for invalid or successful actions.

Without validation and clear feedback, users may accidentally add blank tasks or attempt to complete or delete an item without selecting one. The application addresses these cases with dialog messages.

A small GUI-based task manager is therefore useful for demonstrating how common productivity operations can be handled through event-driven programming.

The Smart To-Do Task Manager addresses these requirements with a Java Swing interface and in-memory task list. It provides the main task operations while keeping the implementation compact and understandable.

2.2 Project Overview

The Smart To-Do Task Manager is a Java desktop application whose main window is created by the SmartTodoGUI class. It uses Swing components to accept task text, select a priority, display the task list, and invoke actions through buttons.

The application supports adding tasks, assigning High/Medium/Low priority, sorting the list by priority, marking a selected item as completed, and deleting a selected item. It also provides validation messages for empty input and missing selection.

The system includes the following functionalities:

• Task entry and priority selection
• Add task with input validation
• Automatic priority-based sorting
• Mark selected task as completed
• Delete selected task
• Dialog-based success and error messages

The supplied implementation uses Java Swing and AWT event handling. Task data is kept in memory for the current session; file or database persistence is outside the current scope.

2.3 Scopes and Limitations

Scope

The scope of the Smart To-Do Task Manager includes the following:

• Creating and displaying tasks
• Assigning one of three priority levels
• Sorting tasks by priority
• Marking a task as completed
• Deleting a selected task
• Providing simple feedback through dialogs

The application may be useful for:

• Students organizing academic work
• Individuals tracking everyday activities
• Users who need a lightweight desktop task list

3. SYSTEM DESIGN

System design defines the layout of the main window, the task data structure, and the actions performed when buttons are clicked. The application uses a single JFrame and separates its responsibilities among input controls, action buttons, and a list model.

The main window contains a title, task text field, priority combo box, Add Task button, Complete button, Delete button, and scrollable task list. Each button is connected to an action listener that performs the corresponding operation.

The project is organized into a small number of functional modules. Each module handles a specific task-management operation and works with the shared list model.

3.1 Modules

The main modules of the Smart To-Do Task Manager are explained below:

1. Task Entry Module

This module accepts a task description through a JTextField. It trims surrounding whitespace and checks whether the field is empty before adding a task.

After a valid entry, the task is added to the list with the selected priority and the input field is cleared.

2. Priority Selection and Sorting Module

The priority combo box provides High, Medium, and Low options. After a task is added, the sortTasks() method arranges list entries according to priority using the getPriority() method.

Priority levels used by the application:

High

Medium

Low

Tasks are displayed in a scrollable list so the user can review the current items.

3. Task Completion Module

This module checks whether a task is selected. If it is not already completed, the application adds a completed label to the task and displays a confirmation message.

4. Task Deletion and Validation Module

The Delete button removes the selected task from the list when a valid selection exists.

If the user has not selected a task, the application displays a message asking for a selection.

The same selection validation is used by the Complete button, and empty task descriptions are rejected by the Add Task operation.

Valid selection → perform the requested action.

No selection or invalid input → display a helpful dialog message.

5. Application Startup and Exit Module

The main() method schedules creation of the SmartTodoGUI window through SwingUtilities.invokeLater(), allowing the GUI to be initialized on Swing's event-dispatch thread.

The main window uses JFrame.EXIT_ON_CLOSE so the application terminates when the user closes the window.

The current implementation does not include a separate save or confirmation-on-exit feature. Tasks remain in memory only while the application is running.

4.Methodology

Methodology describes the stages used to understand, design, implement, and verify the Smart To-Do Task Manager. The work is based on the source code available in the project repository.

The development approach begins with identifying the required task operations, planning the GUI components, implementing event handling, and checking the expected responses for valid and invalid actions.

The development process includes the following stages:

• Requirement analysis
• GUI and module design
• Java program development
• Functional testing
• Results and documentation

4.1. Problem Analysis

The requirements were identified from the intended task-management workflow: enter a task, assign priority, add it to the list, mark it completed, or delete it. Validation and visible feedback are included to help users understand the result of each action.

The main problems identified in manual result processing are:

• Empty task input must be rejected.
• Tasks must retain their selected priority in the displayed entry.
• High priority tasks must be ordered before Medium and Low priority tasks.
• Completion must require a selected task.
• Deletion must require a selected task.
• User actions should produce suitable messages.

These requirements guide the structure of the interface and the actionPerformed() method.

4.2. System Design

The GUI is designed with a JFrame containing labels, a text field, a priority combo box, buttons, and a scroll pane around the task list.

The system is divided into modules such as:

• Task Entry Module
• Priority Selection and Sorting Module
• Task Completion Module
• Task Deletion and Validation Module
• Application Startup and Exit Module

The interface uses action listeners to connect user actions to the underlying task-list operations.

4.3. Program Development

The application is implemented in Java in the SmartTodoGUI class, which extends JFrame and implements ActionListener.

The following steps were followed during development:

• Create and configure the main JFrame.
• Add labels, text field, priority selector, and buttons.
• Store displayed tasks in a DefaultListModel.
• Handle button clicks in actionPerformed().
• Sort tasks using the selected priority.
• Display validation and success messages with JOptionPane.

The source code uses Swing GUI components, AWT event handling, conditional statements, collection sorting, and the Swing event-dispatch mechanism.

4.4. Testing and Validation

Testing checks whether each task operation responds correctly to typical and invalid user actions.

The following representative test cases can be used to validate the application:

Example Test Cases:

Test Case	**Expected Result**
Add valid task	Task added and input cleared
Add empty task	Validation message displayed

Test Case	Expected Result
Complete selected task	Task marked completed
Delete selected task	Task removed from list

These checks describe expected behavior from the supplied code. Actual screenshots can be captured by running SmartTodoGUI.java in a Java environment.

4.5. Result and Documentation

The expected result is a task manager window where users can add tasks with priorities, see them ordered by priority, mark selected tasks completed, and delete selected tasks.

The source repository contains the Java implementation. The input and output illustrations in this report should be captured from the running application if screenshots are required for submission.

This document explains the project purpose, modules, algorithm, test cases, and key implementation details while retaining the supplied report template.

4.6. Algorithm

Algorithm for the Smart To-Do Task Manager:

1. Start the application.
2. Create and display the main task manager window.
3. Enter a task description and select a priority.
4. Click Add Task; validate that the description is not empty.
5. If valid, add the task to the list, clear the field, and sort by priority.
6. For Complete or Delete, check that a task is selected and perform the requested action.
7. Display a suitable feedback message after each operation.

8. Continue until the user closes the application.

4.7. Flowchart

The flowchart should represent the task manager workflow:

Example flow:

Start → Display Task Manager Window → Enter Task and Select Priority → Click Add Task → Validate Input → Add Task and Sort by Priority → Select Task → Complete or Delete → Display Feedback → Repeat or Close Window → End

The flowchart can be drawn using:

- draw.io
- Lucidchart
- MS Visio

5. SOURCE CODE

Complete Java source code (SmartTodoGUI.java):

```
import javax.swing.*;
```

```
import java.awt.*;
import java.awt.event.*;

public class SmartTodoGUI extends JFrame implements ActionListener {
    JLabel titleLabel, taskLabel, priorityLabel;
    JTextField taskField;
    JComboBox<String> priorityBox;
    JButton addButton, completeButton, deleteButton;
    DefaultListModel<String> taskList;
    JList<String> list;
    JScrollPane scrollPane;

    public SmartTodoGUI() {
        setTitle("Smart To-Do Task Manager");
        setSize(500, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(null);
        getContentPane().setBackground(Color.WHITE);

        titleLabel = new JLabel("SMART TO-DO TASK MANAGER");
        titleLabel.setBounds(70, 20, 380, 30);
        titleLabel.setFont(new Font("Arial", Font.BOLD, 19));
        add(titleLabel);

        taskLabel = new JLabel("Enter Task:");
        taskLabel.setBounds(30, 80, 100, 25);
        add(taskLabel);

        taskField = new JTextField();
        taskField.setBounds(130, 80, 300, 25);
        add(taskField);

        priorityLabel = new JLabel("Priority:");
        priorityLabel.setBounds(30, 120, 100, 25);
        add(priorityLabel);

        String[] priorities = {"High", "Medium", "Low"};
        priorityBox = new JComboBox<>(priorities);
        priorityBox.setBounds(130, 120, 150, 25);
        add(priorityBox);

        addButton = new JButton("Add Task");
        addButton.setBounds(30, 165, 120, 30);
        addButton.addActionListener(this);
        add(addButton);

        completeButton = new JButton("Complete");
        completeButton.setBounds(170, 165, 120, 30);
        completeButton.addActionListener(this);
        add(completeButton);

        deleteButton = new JButton("Delete");
        deleteButton.setBounds(310, 165, 120, 30);
        deleteButton.addActionListener(this);
        add(deleteButton);

        taskList = new DefaultListModel<>();
        list = new JList<>(taskList);
        scrollPane = new JScrollPane(list);
        scrollPane.setBounds(30, 220, 400, 200);
        add(scrollPane);
```

```
    setLocationRelativeTo(null);
    setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    if (e.getSource() == addButton) {
        String task = taskField.getText().trim();
        String priority = (String) priorityBox.getSelectedItem();

        if (task.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Please enter a task!");
        } else {
            taskList.addElement(task + " - Priority: " + priority);
            taskField.setText("");
            sortTasks();
            JOptionPane.showMessageDialog(this, "Task added successfully!");
        }
    }
    else if (e.getSource() == completeButton) {
        int index = list.getSelectedIndex();
        if (index != -1) {
            String task = taskList.get(index);
            if (!task.startsWith("[Completed] ")) {
                taskList.set(index, "[Completed] " + task);
                JOptionPane.showMessageDialog(this, "Task completed!");
            } else {
                JOptionPane.showMessageDialog(this, "Task is already completed!");
            }
        } else {
            JOptionPane.showMessageDialog(this, "Please select a task!");
        }
    }
    else if (e.getSource() == deleteButton) {
        int index = list.getSelectedIndex();
        if (index != -1) {
            taskList.remove(index);
            JOptionPane.showMessageDialog(this, "Task deleted!");
        } else {
            JOptionPane.showMessageDialog(this, "Please select a task!");
        }
    }
}

void sortTasks() {
    java.util.ArrayList<String> tasks = new java.util.ArrayList<>();
    for (int i = 0; i < taskList.size(); i++) {
        tasks.add(taskList.get(i));
    }
    tasks.sort((a, b) -> getPriority(a) - getPriority(b));
    taskList.clear();
    for (String task : tasks) {
        taskList.addElement(task);
    }
}

int getPriority(String task) {
    if (task.contains("Priority: High"))
        return 1;
    if (task.contains("Priority: Medium"))
        return 2;
    return 3;
}
```

```
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new SmartTodoGUI());
    }
}
```

6. INPUT AND OUTPUT SCREENS

Screenshots recommended for the final report:

- Main Smart To-Do Task Manager window
- Task description entered with priority selected
- Task list after adding and priority sorting
- Task selected and marked completed, or deleted

Capture each screen from the running application and label it clearly.

Suggested figure labels:

Figure 1: Main Window | Figure 2: Task Entry and Priority | Figure 3: Sorted Task List | Figure 4: Completed Task

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Smart To-Do Task Manager provides a compact desktop interface for basic task organization. Users can enter tasks, assign priority, add tasks to a list, mark selected items as completed, and delete selected items.

Java Swing components and AWT event handling are used to build the interface. The program uses a DefaultListModel for the task list and simple logic to sort entries by priority.

The source code includes validation for empty task descriptions and missing selections. The current implementation stores tasks in memory and does not provide permanent storage.

7.2 Conclusion

The project demonstrates how Java Swing can be used to build a simple and interactive task management application.

It supports the core actions of adding, prioritizing, completing, and deleting tasks while providing clear feedback for common errors.

Possible future improvements include file or database storage, due dates, reminders, search and filtering, editing existing tasks, and a more detailed task status view.

8. REFERENCES

The project documentation is based on the Java source code and the following technical references:

Project Source

• SmartTodoGUI.java – Smart To-Do Task Manager source code in the project repository.

Websites

• https://docs.oracle.com/javase/
• https://docs.oracle.com/javase/tutorial/uiswing/
• https://github.com/shamitha669/smart-todolist